
Lab 11 – List, Set, and Map

Aim

This week is focused on using List, Set, and Map – introducing them with their methods. This builds on your works from Lab 7, 9 and 10. In particular, we will be using the StockPrice class again in the lab tasks.

Tips:

1. Complete the previous lab sheet before you start this sheet.
2. Ask a TA if you do not understand the lab tasks, or you cannot explain the output.

Task 11.1. List as an Instance Variable and addStock

Recall from Lecture 11 that List is a data structure that stores any number of objects in an ordered manner while allowing duplicates; and that there are two implementations for it: ArrayList and LinkedList.

In this task, we are going to use a List object, using a LinkedList as the implementation, to store a collection of StockPrice objects:

- Create a new project Lab11
- Copy the StockPrice class from the previous lab into the project
- Create a new class StockMarket
- Add two instance variables:
 - stockList – a list to store StockPrice objects
 - numStocks – to remember how many StockPrice objects are stored in a certain StockMarket object
- Add a constructor to initialize both instance variables.
How would initialize both the instance variables in the constructor?

Your class should look like the following:

```
import java.util.LinkedList;
import java.util.List;

public class StockMarket {

    List<StockPrice> stockList;
    int numStocks;

    public StockMarket() {
        stockList = new LinkedList<StockPrice>();
        numStocks = 0;
    }
}
```

- Note that it is okay to remove the type parameter in the LinkedList constructor. Why?

- Next, add an instance method `addStock` that takes a `StockPrice` object and add it into the `StockMarket` accordingly!

Solution:

```
public void addStock(StockPrice stock) {
    stockList.add(stock);
    numStocks++;
}
```

We invoke the `add` method of the `List` to add the `StockPrice` object and update `numStocks`.

- We want to be able to print and display a `StockMarket` object, for debugging purposes. Override the `toString()` method so we can `println` a `StockMarket` object nicely!

One possible solution:

```
@Override
public String toString() {
    return numStocks + " stocks\n" + stockList.toString();
}
```

In the solution above, we call the `toString` method of a `List` object which will print all the contents separated by commas and enclosed in square brackets.

- Now let's test our works so far by creating a new `StockMarket` object and adding two `StockPrice` objects into it.

For example:

```
public static void main(String[] args) {
    StockMarket stockMarket1 = new StockMarket();
    StockPrice sp1 = new StockPrice("AAPL", "2007-12-1", 91.8, 92.33,
        90.1, 91.32, 28395700);
    stockMarket1.addStock(sp1);
    stockMarket1.addStock(new StockPrice("AAPL", "2007-12-4", 91.88,
        93.05, 90.5, 91.12, 25340600));
    System.out.println(stockMarket1);
}
```

- The output should show the number of `StockPrice` objects followed by two calls to `StockPrice`'s `toString()` from the previous week, separated by a comma and enclosed by a pair of square brackets.

Task 11.2 deleteFirstStock

A List object must be able to grow and shrink, as a result adding and deleting an object.

Write a method `deleteFirstStock(int index)` that removes the earliest `StockPrice` object that is added into the `StockMarket` object.

Solution:

```
public StockPrice deleteFirstStock() {  
    numStocks--;  
    return stockList.remove(0);  
}
```

Notice that we call the `remove` method of `List` which takes the index of the elements – 0 being the first, and return the deleted element.

Note that you should not forget to update the `numStocks` instance variable as well.

To test, add the following codes into the main method following the codes in the previous lab task:

```
StockPrice deletedStock = stockMarket1.deleteFirstStock();  
System.out.println(stockMarket1);  
System.out.println(deletedStock);
```

Do you get the expected result that the `StockMarket` now contains 1 less `StockPrice` object, and the first added `StockPrice` object getting printed?

As an additional exercise, try yourself to generalize to `deleteStock(int index)` that removes the stored `StockPrice` object at location `index`; and try to provide additional check to `index` and throw an `IndexOutOfBoundsException` object whenever the supplied `index` is not valid.

Task 11.3. All Duplicates with Map

Write a function `allDuplicates` that given a list of string, return true if all strings appear at least twice in the list, and return false if any of the strings is distinct. You have to use Map in your algorithm.

Sample test cases:

```
allDuplicates(["a", "b", "b", "a"]) → true
allDuplicates(["a", "b", "b", "a", "c"]) → false
```

Solution:

```
public static boolean allDuplicates(List<String> list) {

    Map<String, Integer> map = new HashMap<>();
    for (String s : list) {
        if (!map.containsKey(s))
            map.put(s, 1);
        else
            map.put(s, map.get(s) + 1);
    }

    for (String s : map.keySet()) {
        if (map.get(s) < 2)
            return false;
    }
    return true;
}
```

Explanation:

The first enhanced for loop iterates over the list of strings and record each of the string's occurrence using a Map.

The second enhanced for loop iterates over the map by the keys and return false if any value occurs only once (distinct).

The method returns true at the end, since it means all values have duplicates.

Task 11.4 All Distinct with Set

Write a function `allDistinct` that given a list of string, return true if all strings are distinct in the list, and return false if there are some duplicated strings. You have to use Set in your algorithm.

Sample test cases:

```
allDistinct(["a", "b", "c", "d"]) → true
allDistinct(["a", "b", "c", "a"]) → false
```

Solution:

```
public static boolean allDistinct(List<String> list) {

    Set<String> set = new HashSet<>();
    for (String s : list) {
        set.add(s);
    }
    return list.size() == set.size();
}
```

Explanation:

In this simple solution, we leverage the fact that all elements in a set is distinct.

Using an enhanced for loop, we add all elements of the list into the set.

If the set of the size is still the same as the list, we can conclude that all list elements are distinct, and not distinct if otherwise.

This is the end of CPT111 2223 Lab 11 Task Sheet.